

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 1

з дисципліни «Агентно-орієнтоване програмування» на тему:

**«СТВОРЕННЯ АГЕНТІВ У PYTHON ЗА ДОПОМОГОЮ
ФРЕЙМВОРКУ SPADE»**

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Доцент

Олена ПОДКОВАЛІХІНА

1 Мета роботи

Ознайомитися з основами створення агентів у Python. Навчитися використовувати фреймворк SPADE (Smart Python Agent Development Environment) для побудови агентних систем. Реалізувати агентів, які можуть взаємодіяти між собою через обмін повідомленнями.

2 Завдання до лабораторної роботи

Реалізувати агентів для обміну числовими даними та обчислення середнього значення.

3 Виконання лабораторної роботи

Для виконання цієї лабораторної роботи було створено три аккаунти на сервері XMPP. Також було переглянути методичні вказівки та запущено код згідно з прикладами.

Для реалізації застосунку було створено трьох агентів.

Перший буде приймати значення від двох інших. Цей агент буде зберігати загальну суму всіх значень та їх кількість, цього буде достатньо для обчислення середнього значення. В майбутньому поточну логіку можна дописати та зберігати більш детальну інформацію про числа, які в нього приходять.

Інші два агенти будуть генерувати числа від 0 до 10 та відправляти їх до першого. Оскільки агенти два, то вони будуть запускатись паралельно.

В більш складному варіанті виконання цього завдання перший агент повинен запитувати значення у інших агентів, отримувати підтвердження чи вони можуть надіслати інформацію, чекати на неї, а після отримувати повідомлення, що агент завершив прислати інформацію. Така схема буде більш масштабованою та загально більш правильною, але це виходить за рамки поточної лабораторної роботи.

На рисунку 1 показано приклад запуску програми

```
C:\home\university-4\agent-oriented-programming\lbl\.venv\Scripts\python.exe C:\home\univer
Using slower stringprep, consider compiling the faster cython/libidn one.
Receiver: Hi!
Receiver: Waiting for messages!
Sender 1: Hi!
Sender 1: Send message "4"
Receiver: Message from "ltlaitoff-2@xmpp.jp" received! Value = 4
Receiver: New average is 4.0

Receiver: Waiting for messages!
Sender 1: Send message "3"
Receiver: Message from "ltlaitoff-2@xmpp.jp" received! Value = 3
Receiver: New average is 3.5

Receiver: Waiting for messages!
Sender 1: Send message "4"
Receiver: Message from "ltlaitoff-2@xmpp.jp" received! Value = 4
Receiver: New average is 3.6666666666666665

Receiver: Waiting for messages!
Sender 1: Send message "3"
Receiver: Message from "ltlaitoff-2@xmpp.jp" received! Value = 3
Receiver: New average is 3.5

Receiver: Waiting for messages!
Sender 2: Hi!
Sender 2: Send message "9"
Receiver: Message from "ltlaitoff-3@xmpp.jp" received! Value = 9
Receiver: New average is 4.6
```

Рисунок 1 – Запуск програми

4 Тест розробленої програми

```
import random
import spade
```

```

from spade.agent import Agent
from spade.behaviour import CyclicBehaviour
import asyncio
from spade.message import Message
from spade.template import Template

import keys

int_message_template = Template()
int_message_template.set_metadata("performative", "inform")
int_message_template.set_metadata("language", "FIFA-SL")
int_message_template.set_metadata("ontology", "INTEGER")

class AverageNumbersAgent(Agent):
    # Custom agent identifier. For debug purposes only
    name = "Unknown receiver"
    message_time = 5
    sum = 0
    count = 0

    def __init__(self, jid: str, password: str, name: str):
        super().__init__(jid, password)
        self.name = name

    class HandleIntBehaviour(CyclicBehaviour):
        async def run(self):
            print(f"{self.agent.name}: Waiting for messages!")
            message = await self.receive(timeout=self.agent.message_time)

            if not message:
                print(f"{self.agent.name}: Bye!")
                await self.agent.stop()
                return

            assert int_message_template.match(message)
            print(f"{self.agent.name}: Message from \"{message.sender}\" received! Value = {message.body}")

            value: int = int(message.body)
            self.agent.new_value(value)

        def new_value(self, value: int):
            self.sum += value
            self.count += 1

            average = self.average()
            print(f"{self.name}: New average is {average}\n")

        def average(self):
            return self.sum / self.count

        async def setup(self):
            print(f"{self.name}: Hi!")
            self.add_behaviour(self.HandleIntBehaviour(), int_message_template)

class GeneratorAgent(Agent):
    # Custom agent identifier. For debug purposes only
    name = "Unknown sender"

    # ID of receiver
    to_id = ""

    def __init__(self, jid: str, password: str, name: str, to_id: str):
        super().__init__(jid, password)
        self.name = name

```

```

        self.to_id = to_id

class SendBehaviour(CyclicBehaviour):
    count = 0

    def generate_int_message(self) -> Message:
        value = random.randint(0, 10)

        message = Message(to=self.agent.to_id)
        message.body = str(value)
        message.set_metadata("performative", "inform")
        message.set_metadata("language", "FIFA-SL")
        message.set_metadata("ontology", "INTEGER")
        assert int_message_template.match(message), "Int message should match int
template"

        return message

    async def run(self):
        message = self.generate_int_message()

        await self.send(message)
        print(f"{self.agent.name}: Send message \"{message.body}\"")

        if self.count >= 10:
            print(f"{self.agent.name}: Bye!")
            await self.agent.stop()

        await asyncio.sleep(1)
        self.count += 1

    async def setup(self):
        print(f"{self.name}: Hi!")
        self.add_behaviour(self.SendBehaviour())

async def main():
    receiver_agent = AverageNumbersAgent(keys.XMPP_ID_ONE, keys.XMPP_PASSWORD_ONE,
"Receiver")
    await receiver_agent.start()

    sender_agent_one = GeneratorAgent(keys.XMPP_ID_TWO, keys.XMPP_PASSWORD_TWO, "Sender 1",
keys.XMPP_ID_ONE)
    sender_agent_two = GeneratorAgent(keys.XMPP_ID_THREE, keys.XMPP_PASSWORD_THREE, "Sender
2", keys.XMPP_ID_ONE)

    await sender_agent_one.start()
    await sender_agent_two.start()

    await spade.wait_until_finished(receiver_agent)

if __name__ == "__main__":
    asyncio.run(main())

import os
from dotenv import load_dotenv
load_dotenv()

XMPP_ID_ONE=os.getenv('XMPP_ID_ONE')
XMPP_PASSWORD_ONE=os.getenv('XMPP_PASSWORD_ONE')
XMPP_ID_TWO=os.getenv('XMPP_ID_TWO')
XMPP_PASSWORD_TWO=os.getenv('XMPP_PASSWORD_TWO')
XMPP_ID_THREE=os.getenv('XMPP_ID_THREE')
XMPP_PASSWORD_THREE=os.getenv('XMPP_PASSWORD_THREE')

```

```
assert XMPP_ID_ONE is not None and XMPP_ID_ONE != "", "XMPP id ONE environment variable is not defined!"
assert XMPP_PASSWORD_ONE is not None and XMPP_PASSWORD_ONE != "", "XMPP password ONE environment variable is not defined!"
assert XMPP_ID_TWO is not None and XMPP_ID_TWO != "", "XMPP id TWO environment variable is not defined!"
assert XMPP_PASSWORD_TWO is not None and XMPP_PASSWORD_TWO != "", "XMPP password TWO environment variable is not defined!"
assert XMPP_ID_THREE is not None and XMPP_ID_THREE != "", "XMPP id THREE environment variable is not defined!"
assert XMPP_PASSWORD_THREE is not None and XMPP_PASSWORD_THREE != "", "XMPP password THREE environment variable is not defined!"
```

5 Висновки

Я ознайомився з основами створення агентів у Python. Навчився використовувати фреймворк SPADE (Smart Python Agent Development Environment) для побудови агентних систем. Реалізував агентів, які можуть взаємодіяти між собою через обмін повідомленнями.